KV cache/prefix cache 感知的智能调度 routing.

这是 llm-d 最有区分度的地方。Router 通过 Gateway API Inference Extension 接入，根据各实例的前缀缓存命中情况、队列长度，KVcache 占用来选路由目标。在多轮对话、Agent、RAG 这类前缀高度重复的负载下，对 TTFT 和吞吐 throughput 的改善最直接。

第一节 KV cache & prefix cache.

1. 模推理 inference 分两个阶段

- prefill：把输入的整段 prompt 一次性过一遍 model。每一层 attention 都会为每个 token 算出一对向量，叫 K(key) 和 V(value).
- decode：一个 token 一个 token 地往外生成，每生成一个新 token，都要和前面所有的 token 的 KV 做 attention matmul.

第二节 为什么 consistent hashing 还不够

2. Routing 过程，为什么 consistent hashing 不够.

① 哈希只管"这个会话发去哪台"，而不管那台机在忙不忙。有的会话是重度 agent，上下文十几万 token，request 频繁，正好几个哈希到同一台，那台的队列就会堆起来。排队等的时间，可能比省下的 prefill 的时间还多.

② 不知道缓存的真实状态，显存紧张时，vLLM 会按 LRU 淘汰 block 级逐出。哈希作出后很久会将此会话发到那台机器，但 cache 可能早就没了，被驱逐了。

第三节 llm-d调度器的架构

①请求走的路径

客户端

↓

API Gateway（基于Envoy的网关）

↓①网关把请求交给EPP，问"该发给谁？

EPP（Endpoint Picker，也就是调度器）

↓②按各个vLLM Pod的指标和缓存状态，选出一个pod

Gateway把请求转发到被选中的vLLM Pod.

主要的选择因素④

prefix-cache-scorer / queue-scorer / kv-cache-utilization-scorer / token-load-scorer

---

Envoy和ext_proc

Envoy是一个开源高性能代理（proxy），它工作在第七层（HTTP）层，能看懂HTTP请求，做路由，load balance，retry，限流，TLS。

GCP的API Gateway load balancer就是拿Envoy做的，

所以在k8s的Gateway API生态里，大部分网关实现其实底层是一个Envoy+一个控制面（control plane）.

ext_proc Envoy处理请求时有一条过滤器链（filter chain），每个过滤器都可以检查或修改请求。ext_proc（external processing）是其中一条特殊的过滤器，它会把请求的header，body通过gRPC流式发给一个外部服务，由外部服务决定要怎么改这个请求，Envoy再照着做。

在 llm-d 里具体怎么配合：☆⑧.

①猎求到达 Envoy　　filter (chain)

②☆ Envoy 与 ext-proc 过滤器把 header + body (含 prompt、模型名) 通过 gRPC 发给 EPP

③ EPP 解析 prompt，结合各 pod 的指标打分，选出目标 pod.

④ EPP 返回结果：在猎求上加一个 header，写明目标 pod 的地址.

⑤ Envoy 读到这个 header，直接把猎求转发给那个 pod.

LoRA?

— Low-Rank Adaption 是一种轻量的模型微调方法.
一个基础 base model 可以同时挂多个 adapter，vLLM 支持 multi-LoRA serving，同一个 pod 里只加载一份基础 model，但不同的 req 可以指定不同的 adapters. 但是，每个 pod 同一时间能激活的 adapter 数量有限. 假设 req 要用 xyz adapter，若 pod 有 xyz adapter，则 OK，直接跑，但若 vLLM 那个 pod 未加载 adapter，则要从存储 disk driver load adapter xyz，可能还要 replace 别的 adapter，↑ latency. 所以 EPP 的 filter 阶段 可以先 filter 掉未 pre-load 某 adapter 的 pod.

8

第四节 EPP怎么知道每个pod缓存了什么.

EPP是个独立的服务，EPP它看不到vLLM显存里的内容. 要回答"podA上有当前这个猜出的多少prefix?" 只有两种方法:

① 自己猜，根据自己过去把哪些猜出发去了哪里来推断.

② 让vLLM告诉它: vLLM pod每次缓存变化，主动汇报，(precise mode.)

(Approximate 模式).

① 自己猜:

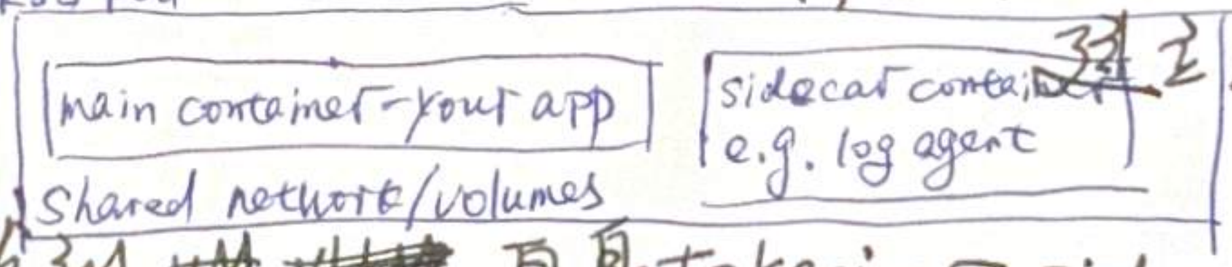
(i) EPP本身不带tokenizer，把prompt切成固定大小块，构造hash.

(ii) EPP在内存里维护一个LRU index，记录最近哪些prefix hash被发往哪些pod.

(iii) 打分 scorer 按命中块数/prompt总块数.

这种方法不需要tokenizer sidecar，也不需要和vLLM/model pod建立ZMQ连接.

② precise mode.



(i) EPP把prompt发到自身tokenizer sidecar，拿到tokens.

(ii) vLLM pod当内部KV cache发生变化，通过ZeroMQ发布出KVEvents.

(iii) KV cache Indexer订阅这些event，maintain一份token-to-pod in table.

---

gRPC:

RPC为remote procedure call，让你调用另一台机器上的函数，写起来就像调用本地函数一样

第二项 P/D disaggregation

即使有 chunked prefill，也要有 P/D disaggregation.

（做章机上io mitigation）

第二节分离io代价，KV cache要频繁.

① 举个例子，prefill io计算量约为 2×参数量×token数 = ~~2×700Fo×~~ 2×70B×8k ≈ 1.1×10^15 FLOPs

② 一块H100 io FP16 peak ≈ 1×10^15 FLOP/s，但实际经跑到一半左右.

也就是说，4张卡并行，prefill大约 0.5s

第三节 KV 怎么搬，RDMA 和 NIXL

① RDMA (Remote Direct Memory Access)，让网卡直接读写另一台机器io内存，绕过双方io操作系统 kernel 和 CPU.

OS

发送端内存 → 网卡 → 网络 → 网卡 → 直接收端内存.

(NIC)

3个关键特性：

- kernel bypass：应用直接操作网卡(NIC - network interface card)，不走系统调用.
- zero-copy：data 从内存直接到网卡，没有中间副本.
- one-sided：一方可以直接读/写对方内存，对方CPU完全不用参与，甚至不知道这次读写发生了.

常见io RDMA网络：InfiniBand.

为什么需要 NIXL？

实际环境情况非常复杂：

- 传输方式：InfiniBand, NVLink, RoCE, TCP...
- 存储位置：GPU显存，CPU内存，远端存储.....

RDMA over Converged Ethernet：A way to run RDMA over an Ethernet network.

NIXL（NVIDIA Inference Xfer Library）是 Nvidia 开源的数据传输库，专门为推理场景设计，为了"把这N块内存从A搬到B".

在vLLM和llm-d怎么用它？

vLLM 有一个 KV connector abstract layer，专门负责 KV cache 怎样进出这个 vLLM instance. 其中 NIXLConnector 就是用 NIXL 在两个 vLLM instance 之间传 KV，llm-d 的 P/D disaggregation 就是建立在它之上.

llm-d	决定：哪个请求要分离，发给哪个P，哪个D？	调度层.
vLLM	NixlConnector：管理哪些KV block要传.	引擎层.
NIXL	统一的搬数据接口	传输抽象层.
UCX	走 RDMA / NVLink / 或TCP	传输实现层.
网卡/GPU	GPUDirect RDMA 等硬件能力	硬件层.

第4节 llm-d里一个请求在P/D架构下怎么走.

1. 参与者

- Envoy网关：和第一项一样
- EPP：config成 disagg-profile-handler mode，负责选出一对P和D.
- 一个 InferencePool：P和D都在同一个池里，靠 pod 的 label 区分，#Hidden prefill decode prefill-decode
- 对于 decode pod，里面除了 vLLM，还多了一个 routing proxy sidecar.

2. profile Handler：EPP 的"分拣台"

第1项讲过 EPP 的 Filter → score → Pick. 这里多了一层 profile handler:

- 默认的 single-profile-handler 一套 profile，出一个 pod，就是第一项的普通 mode.
- disagg-profile-handler：

  有 decode-profile 和 prefill-profile 两套，分别用 Filter 只留下对应的 label 的 pods，再各自 score 挑选.

3. 为什么需要 sidecar?

EPP 只负责"做决定"，Envoy 一次只能把请求转发到一个目的地，而 P/D 是一个多步骤的过程. 所以先找到 D，若需要，让 D 先把 prefill req 到 P，再回 D.

完整流程.

用户 → Envoy —(ext_proc)→ (gRPC streaming to) EPP

EPP:
① 先跑 decode-profile，选出 D
② 问 decider："这个 prompt 在 D 上已经 cache 了多少？"
- 多 → 不 disagg → 只返回 D.
- 多 → 再跑 prefill-profile 选出 P.

  若：prompt 长吗？若很短，也可只返回 D.

③ 返回 D，若 disagg，则再加一个 header：
x-prefiller-host-port: <P's address>.

Envoy → D's Routing sidecar

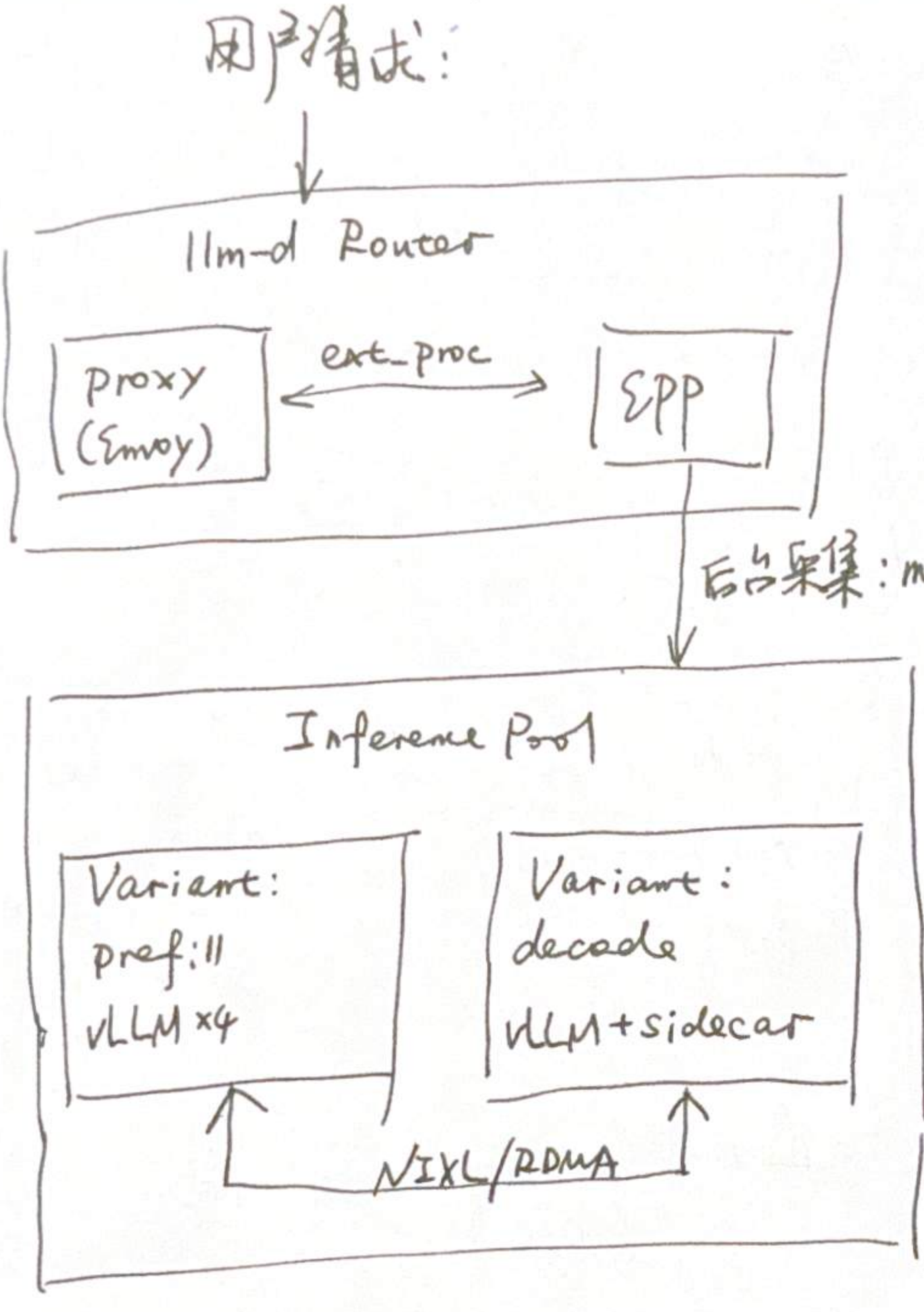
有 x-prefiller-host-port 吗？
- 无 → 直接转给本地 vLLM，普通 inference.
- 有 → 走 P/D

④ 改写请求发给 P（leader?）的 kv_transfer_params.
⑤ P 返回 kv_transfer_params（你要 pull 的 KV block 在我这边，id 为... index）
⑥ 把这些参数塞进原请求，转给本地 vLLM (D)
⑦ D 向 NixlConnector 通过 NIXL pull P's memory's KV blocks.
⑧ D starts to decode, token stream output.

example:
```
{"do_remote_prefill": true,
 "remote_engine_id": "prefill-pod-2",
 "remote_request_id": "req-p-abc123",
 "remote_host": "10.20.3.17",
 "remote_port": 5600,
 "remote_block_ids": [81, 82, 83, 84]}
```

sidecar → Envoy → 用户

第3项：llm-d的整体架构，以及它和vLLM的分工



llm-d 和 vLLM 如何分工：

① consistent batch processing, paged attention, chunked prefill:	vLLM
② single instance's prefix hashing	vLLM
③ tensor parallelism, expert parallelism in single instance	vLLM
④ cross instance's prefix routing	llm-d
⑤ 全局 KV-cache Indexer	llm-d
⑥ 要不要 P/D，选哪对 P 和 D？(EPP + decider)	llm-d
⑦ 编排 P/D 两阶段请求 (D上面is routing sidecar)	llm-d
⑧ KV block is实际性输	vLLM (NixlConnector)
⑨ 扩缩容is决策 →	llm-d (EPP导出is metrics)

两者之间的接口

接口

~~kvCacheIndexer~~ 方向

Prometheus metrics (队列长度, KV占用) vLLM → EPP

KV Event (ZMQ) (prefix cache usage) vLLM → KVIndexer

render (精确分词) EPP → vLLM

kv_transfer_params (HTTP请求体里的参数) sidecar ⟷ vLLM

KV connector (NIXLconnector) vLLM内部 → NIXL

EPP把 prompt 发到 vLLM 的 HTTP render API (/v1/completions/render), 通常是 EPP pod 里的一个 render sidecar, 为拿到精确的 token ID.

L7: HTTP/gRPC level (RUST API)

只懂TCP/UDP level (ip).

(round-robin)

长什么叫 "k8s 原生"

- InferencePool 对标 service, 普通的 k8s Service 只做 L4 的轮询, 不懂 LLM. InferencePool 保留了"用标签选 Pod"这个熟悉的模型, 但它的流量交给懂 LLM 的 Router 来分配.
- 用标签而不是新资源表达角色: Variant 就是 pod label, P 和 D 就是两个普通的 deployment.
- 复用 k8s 生态的标准组件: 首先 Gateway API 管入口, 然后好的容器也不造轮子, 文档说是由 KEDA 的 prometheus scaler 读取 EPP 导出的指标. (比如队列深度, pool 饱和度, token 积压, 相对 SLO 的预估延迟), 再去创建和管理 HPA, 而无需自定义控制器.

k8s 原生，再深入一点

1. 先理解 k8s 本身的个核心思想.

① 声明式API (Declarative API)：你不告诉 k8s "去启动3个容器"，而是写一份 YAML 声明 "我想要的状态是3个副本".

② 控制器与调和循环 (reconcile loop)：k8s 中有很多控制器，每个都在不停地做同一件事：

While True:
  当前有N个pod?
  对比期望状态 (YAML里写的是N个?) ← deployment
  少了就创建，多了 cool down 一会儿再删.

③ CRD — 自定义资源：k8s 允许你注册新的资源类型，扩展其API.
  注册之后，新资源和 pod、Deployment 一样可以用 kubectl 操作，用 YAML 声明.

④ label & selector：给资源贴标签，其他组件用 selector 按 label 找到。
  (Pods 可以有 one-to-many labels，deployment yaml 声明它.)

2. 原生 (k8s native) 什么意思？

怎么知道有哪些 inference instance？ 原生 llm-d 作法：

实例挂了怎么办？ k8s Health detector (probes 探针 (readiness probe)) 发现 pod 为 Not Ready，EPP 自动把它移出候选.

EPP 通过 k8s API 监听符合 InferencePool label 的 pod.

怎么部署/升级？ 普通 k8s deployment，滚动更新.

MoE 是什么?

- 之前的例子 - Llama-3-70B 是一个 dense model, 意思是每个 token 都要经过 model 的全部 70B params.

Transformer 的每一层主要由两部分组成:

- Attention layer: 让 token 之间相互看, KV cache 就是它产出.
- FFN/MLP layer: 让 token 单独做变换, 可以理解为 model 存储"知识"的主要地方.

- 开 MoE model 例子: deepseek-V3
  - 总 params: 671B → FP8 is 671GB, H100 都没那么大 memory, 必须多机.
  - 每个 token 实际激活 params: 约 37B.
  - 每个 MoE 有 256 个 experts + 1 个共享 expert.
  - 每个 token 从 256 里选 8 个 + 1 个共享 expert.

- TP 因为 all-reduce 的原因, 通信频繁, 所以通常只在一台机器内用 (走 NVLink)
  (all-reduce 汇总结果)

- 每个 MoE layer 都要来回两次 all-to-all:
  - dispatch: 把 token 送到 expert 所在的卡.
  - combine: 把专家算完的结果送回 token 原来所在的卡.

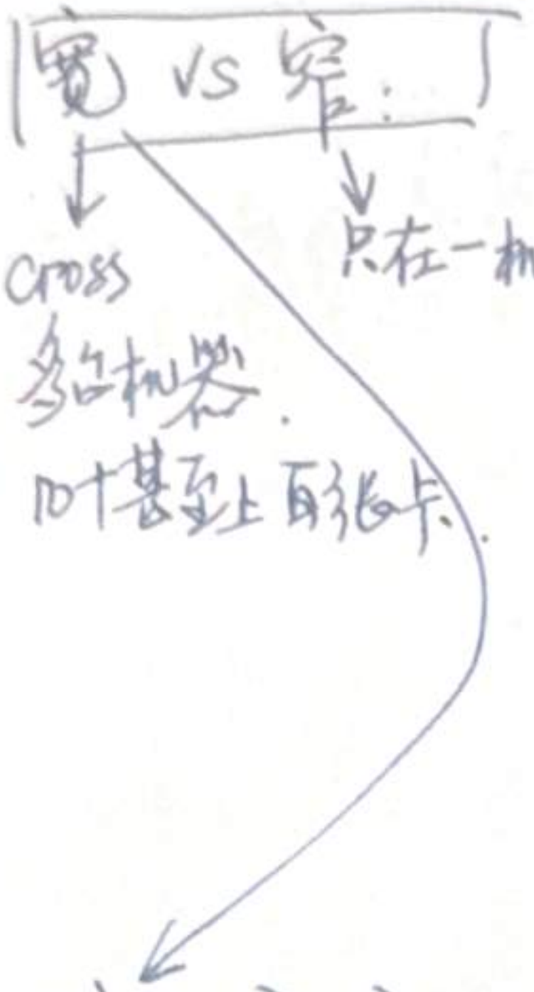
为什么 attention 用 DP 而非 TP?

① attention layer 部分 params 相对较小, 主要是 MoE params, attention params 每卡 copy 一份, 代价不大.

② DeepSeek 用的 MLA, TP 用 head 切, 每张卡负责一部分 head 和对应的 KV.

③ TP 跨机太贵, TP 每层都要 all-reduce, 跨机代价太高很.

	8卡	32卡
每卡 expert 权重(FP8)	约 8# GB	约 21 GB
80 GB 的 H100 还剩多少留给 KV cache?	~~没法带不下~~	约 50 多 GB
系统能承载的并发情况	×	很多
每个 expert 每步能分到 token	×	多,吃不饱.



代价:通信, cross host all-to-all.

- Deepseek-v3 在跨机时,只发一次到那台机器,再在机器内部通过 NVLink 分给 8 张卡.
- EPLB: Expert parallelism Load Balancer

问:对 D 新 gen 的 token,要不要 cache 一份送去 P?

答:会初始在 KV Indexer,不会主动送.

- 不知道有无下一轮
- 不知道什么时候来
- 不知道去哪个 P
- 下一轮可能根本不走 P,在 D 上直接 prefill 短 prompts 了.